Compression-Resilient DCGAN-Based Image Steganography

An Extension of Zain's DCGAN Steganography System

Technical Report - CSC 616: Cybersecurity

University of Miami | April 2026

Abstract

This report presents an extension of an existing DCGAN-based image steganography system that encodes hidden text messages into the latent space of a generative adversarial network, producing synthetic face images from which the original message can be decoded. The baseline system (Zain, 2024) performed well under clean, uncompressed conditions , achieving 100% bit accuracy , but collapsed under JPEG compression, falling to only 58–59% bit accuracy, barely above random guessing. This work identifies that real-world failure mode, proposes a solution, implements it, and demonstrates measurable improvement. By integrating JPEG compression simulation directly into the training loop, the new model achieves 83% bit accuracy at QF=75 and 80% at the more aggressive QF=30, representing a 22–24 percentage point improvement in compression resilience across all tested conditions.

Building on this foundation, four error correction approaches were systematically evaluated : Hamming(7,4) coding, confidence-weighted decoding, a Learned Latent Denoiser, and 3x bit repetition coding , each testing a different hypothesis about the structure of JPEG compression errors in this latent space. Three approaches failed for distinct and informative reasons; 3x repetition coding emerged as the decisive winner, achieving 94.75% bit accuracy and 26% exact message recovery under ideal conditions and 92.50% bit accuracy and 10% exact recovery under aggressive QF=30 compression..

---

1. Motivation

Since the first secret was worth keeping, humanity has found ways to hide it. Ancient Greeks tattooed messages on the shaved heads of slaves and waited for their hair to grow back. Julius Caesar shifted letters by three positions to confuse his enemies. Mary Queen of Scots encrypted her correspondence with conspirators in the Babington Plot to assassinate Queen Elizabeth I - and when her cipher was broken, it sent her to the

executioner's block. In the Second World War, German intelligence reduced entire pages of text to microdots the size of a period and concealed them in plain sight on postcards crossing enemy lines. At the same time, Nazi Germany's Enigma machine - an electromechanical cipher device of staggering complexity - encoded military communications that Allied forces were desperate to read. It took Alan Turing and his team at Bletchley Park, building a machine to fight a machine, to crack it - a breakthrough that historians credit with shortening the war by years. From Leonardo da Vinci's mirror-script notebooks to the dead drops of Cold War spies, the desire to communicate without being understood is as old as communication itself.

The history of secret communication is, in many ways, the history of human ingenuity under pressure.

All of these efforts shared one goal: keeping the message from the wrong eyes. Some achieved it through encryption - scrambling the meaning. Others, like the microdot, through steganography - hiding the very existence of the message. This project lives in that second tradition.

Today, that pressure has migrated to the digital world. As communication increasingly occurs over monitored, filtered, and algorithmically inspected channels, the ability to conceal the existence of a message - not merely its content - has become a meaningful security objective. Encryption protects what is said; steganography protects whether anything was said at all.

Modern platforms routinely apply lossy JPEG compression to user-uploaded images on social media, messaging apps, and cloud storage. This compression degrades or destroys hidden payloads embedded using classical steganographic techniques, and - as the baseline results in this work confirm - it breaks deep learning-based steganographic systems that were not designed with compression in mind. Addressing this fragility is the central contribution of this project.

---

2. Problem Characterization

2.1 The Baseline System and Its Limitation

This project begins with a fork of an existing DCGAN steganography implementation built by Zain. The forked repository provided the Generator and Discriminator architecture, the CelebA training pipeline, and a baseline encoding/decoding scheme that achieved 100% bit accuracy under clean conditions. Everything described in this report - the compression vulnerability identification, the JPEG-aware training loop, the Decoder network, the modular script architecture, and the complete error correction investigation - represents the author's independent contribution built on top of that foundation.

The baseline system (Zain's DCGAN steganography implementation) encodes a text message into the 100-dimensional latent vector z of a DCGAN Generator, producing a 64×64 synthetic face image. A separately trained Decoder CNN reverses the process, recovering the original message from the image. Under clean, uncompressed conditions, the baseline achieves 100% bit accuracy - confirming that the GeneratorDecoder pair successfully learns to preserve latent information in the image domain.

However, real-world image transmission almost always involves JPEG compression. Testing revealed that when baseline-generated images were subjected to JPEG compression before decoding, bit accuracy fell to approximately 58–59% - barely above the 50% rate expected from random guessing. The system was effectively nonfunctional in any realistic transmission scenario.

Core finding: The baseline Decoder learned features that relied on pixel-perfect image fidelity - features that JPEG's lossy DCT quantization destroys. The system had a critical real-world vulnerability.

2.2 Problem Statement

Given a short text message m, generate a synthetic image I such that a trained Decoder D can recover m from I even after I has been subjected to lossy JPEG compression at a range of quality factors. The system must satisfy the following constraints:

- Message capacity: The latent vector z has dimension nz = 100, supporting approximately 12 ASCII characters (100 bits / 8 bits per character).
- Image realism: Generated images must be visually plausible to avoid suspicion, requiring the Generator to simultaneously satisfy the Discriminator (realism objective) and the Decoder (message preservation objective).
- Compression resilience: The Decoder must recover messages from images that have undergone lossy JPEG compression - a non-differentiable transform that must be simulated within the training loop.
- Generalization: Compression resilience must generalize beyond the training compression level. A model that only survives QF=75 compression would not be useful in the real world.

---

3. Technical Approach

3.1 The Core Idea

The baseline Decoder failed on compressed images because it had never seen compressed images during training. The solution is straightforward in concept: train the Decoder on compressed images. By simulating JPEG compression inside the training loop and passing compressed images to the Decoder during each training step, the

Decoder is forced to learn features that survive compression artifacts. The Generator, trained jointly, learns to produce images whose encoded latent information resides in compression-robust image components.

3.2 Architecture Overview

The system uses a three-network architecture trained jointly on the CelebA face dataset:

- Generator (G): Upsamples a 100-dimensional latent vector z to a 64×64 RGB face image via five transposed convolutional layers with BatchNorm and ReLU activations, followed by a Tanh output layer.
- Discriminator (D): Classifies real CelebA images vs. Generator outputs using five convolutional layers with LeakyReLU activations and a Sigmoid output.
- Decoder (D_u): A custom CNN that maps a 64×64 RGB image back to a 100dimensional vector approximating the original z. Two convolutional layers followed by a fully-connected layer with Tanh output. Trained jointly with G.

3.3 Message Encoding Scheme

A text message is converted to a binary string at 8 bits per ASCII character. Each bit is mapped to a float value: 1 → +1.0, 0 → -1.0, producing the 100-dimensional latent vector z. The Generator maps z to a face image; the image is the message. No additional payload is embedded - the image itself encodes the information in its latent structure.

3.4 JPEG-Aware Training Loop

The primary technical contribution is the jpeg_compress_decompress() function integrated into the training loop. During each training batch, after G generates a fake image, the image is passed through this function before being fed to D_u:

1. The normalized image tensor is converted from [-1,1] to a PIL image [0,1]
2. The PIL image is encoded to JPEG at a fixed quality factor (QF=75)
3. The JPEG is decoded back to a tensor
4. The tensor is re-normalized to [-1,1] for the Decoder

The Decoder's reconstruction loss is computed on this compressed image, not the clean original. This is the mechanism that builds compression resilience into both the Decoder and the Generator over 30 epochs of training.

3.5 Loss Functions

Three losses govern the training objective:

- Discriminator loss (BCELoss): Standard adversarial loss separating real CelebA images from generated images.
- Generator adversarial loss (BCELoss): Generator attempts to fool the Discriminator into classifying its outputs as real.

- Reconstruction loss (MSELoss): The Decoder's recovered $\hat{z}$ is compared to the original z, weighted by $\lambda = 50.0$.

Total Generator + Decoder training loss = errG + (50.0 × errRecon). The high λ weight reflects the primary objective: compression-resilient message recovery takes priority over photorealism. This is the correct engineering tradeoff for a steganographic application.

3.6 Training Configuration

Parameter	Value
Base Architecture	Zain's DCGAN (forked)
Epochs	30
Batch Size	128
Latent Vector Dimension (nz)	100
Learning Rate	0.0002
Optimizer	Adam ($\beta1 = 0.5$)
Reconstruction Weight (λ)	50.0
JPEG Quality During Training (QF)	75
Dataset	CelebA (aligned face images, 200K+)
Image Resolution	64 × 64 RGB

---

4. Solution Description

4.1 End-to-End Pipeline

5. Input: A text message string (max ~12 ASCII characters)
6. Encoding: message_to_noise() converts the string to a binary 100-dimensional latent vector z (+1.0 / −1.0 per bit)
7. Generation: G(z) produces a 64×64 synthetic face image
8. Transmission: The image is shared through a channel that may apply JPEG compression
9. Decoding: D_u(image) recovers the predicted latent vector $\hat{z}$ from the compressed image

10. Output: noise_to_message() converts $\hat{z}$ to text by thresholding at 0.0 and converting 8-bit groups to ASCII characters

4.2 Code Contributions

The following files were written or significantly extended as part of this work. The forked baseline provided only the core Generator/Discriminator architecture and training loop; all items below are original contributions:

File	Description
dcgan_based_trainer_v2_4.py	Training loop extended with jpeg_compress_decompress() and a jointlytrained Decoder network
dcgan_based_trainer_v3.py	Refactored trainer with cleaner structure, multi-worker DataLoader support, and improved portability
generate_stego_image.py	Standalone modular script: encodes a text message and generates the corresponding steganographic image
decode_stego_image.py	Standalone modular script: decodes a message from a given image file
decode_image_v2.py	Simplified self-contained decoder for deployment; no external imports required
compress_dataset.py	Dataset preprocessing utility: applies JPEG compression to CelebA training images
compressiontest.py	Automated test: 50 random 12-character messages, each encoded and decoded after JPEG compression at QF=30
non-compressiontest.py	Automated test: 50 random 12-character messages, encoded and decoded from clean PNG (no compression)

4.3 Key Design Decisions

- Training at QF=75, testing at QF=30: The gap between training and test compression levels was intentional. A model that learns only to survive QF=75 might overfit to that specific compression level. Testing at QF=30 - a significantly more aggressive setting - evaluates whether compression resilience genuinely generalizes. The results confirm that it does.
- High reconstruction weight (λ=50.0): Reconstruction loss dominates the training objective, ensuring message recovery is prioritized over image realism. This is the correct tradeoff for steganography.

- Tanh decoder output: Matches the +1.0/−1.0 bit encoding convention, enabling clean binary threshold decoding at 0.0 without additional calibration.
- Modular script architecture: Separating generation and decoding into importable functions enables the test scripts to run without modification and allows deployment with only the trained model weights - no training infrastructure required on the receiving end.

---

5. Experiments, Error Correction, and the Edison Principle

Thomas Edison, when asked about his thousand failed attempts to build a working lightbulb, famously replied: “I have not failed. I have successfully discovered a thousand ways that do not work.” This section documents five distinct experimental phases undertaken to improve message recovery under JPEG compression. Each approach that did not work taught something specific and pointed toward what should come next. The conclusion - that 3x repetition coding is the most effective error correction scheme for this specific system - is supported by four independent lines of evidence, each approaching the problem differently.

5.1 Phase 1: Baseline Compression Testing

The first experiment established the scope of the problem. Both the baseline model (Zain’s) and the author’s new compression-aware model were evaluated under three conditions: no compression, JPEG QF=75, and JPEG QF=30.

Test Condition	Baseline (Zain’s)	New Model (This Work)	Improvement
No compression (PNG)	100%	83%	−17% (expected tradeoff)
JPEG QF=75	59%	83%	+24 percentage points
JPEG QF=30	58%	80%	+22 percentage points

The new model’s compression-aware training produced a 22–24 percentage point improvement under real-world compression conditions, at a modest and expected cost to uncompressed performance. With ~17–20% of bits still being flipped by JPEG, the question became: can error correction recover the remaining accuracy?

5.2 Phase 2: Hamming(7,4) Error Correction

Hypothesis: Hamming(7,4) encodes every 4 data bits as 7 bits (adding 3 parity bits), enabling correction of any single-bit error per block. Applied on top of the existing encoding, it should improve message recovery without retraining.

Result: Hamming produced only 0.7–1.9 percentage point improvement in bit accuracy and near-zero exact match rates. Under QF=30, Hamming actually performed worse than standard encoding (80.50% vs 81.33%).

Diagnosis: Hamming corrects exactly one bit error per 7-bit block. JPEG compression does not produce isolated single errors - it damages bits randomly and densely across the entire image, causing most 7-bit blocks to sustain more than one error. When two bits in a block are wrong, Hamming's parity logic miscorrects, making the result worse. Finding: JPEG produces multi-bit errors per block, exceeding Hamming's correction capacity.

5.3 Phase 3: Confidence-Weighted Decoding

Hypothesis: The Decoder network's output magnitude carries information about certainty. Values close to the 0.0 decision boundary are uncertain and likely wrong. By identifying these bits and applying adaptive strategies - ensemble voting across multiple thresholds, neighbor-context correction, adaptive threshold derived from output distribution - message recovery should improve without any encoding overhead.

Three strategies were implemented and tested: adaptive thresholding, confidencefiltered decoding with neighbor context, and ensemble voting across five threshold values.

Result: All three confidence strategies produced identical results to standard fixedthreshold decoding. Bit accuracy and exact match rates were unchanged across all compression conditions.

Diagnosis: Confidence weighting requires uncertain bits - latent values close to 0.0 - to have something to work with. This Decoder produces predominantly high-confidence outputs (values strongly positive or negative), with very few bits landing near the boundary. When all five ensemble thresholds agree, there is nothing to correct.

Finding: The Decoder's high-confidence output profile leaves no uncertain bits for softdecision strategies to exploit.

5.4 Phase 4: Learned Latent Denoiser

Hypothesis: Unlike hand-crafted error correction, a neural network trained on thousands of (clean z, noisy $\hat{z}$) pairs should learn the specific statistical structure of JPEG errors in this model's latent space and outperform all fixed schemes.

Implementation: 20,000 training pairs were generated - 10,000 at QF=75 and 10,000 at QF=30 - by running the trained Generator and Decoder forward with JPEG compression applied in between. A Corrector network (5 fully-connected layers, 600,548 parameters,

residual connection, Tanh output) was trained for 50 epochs on these pairs using MSE loss with Adam optimizer.

Result: The Corrector's best validation performance was 74.45% bit accuracy at epoch 3 - worse than the standard decoder's 86%+. The network never improved meaningfully beyond this and was correctly saved at its epoch 3 peak. In the five-way comparison, the Learned Denoiser matched or slightly underperformed standard decoding across all conditions.

Diagnosis: A neural network can only learn patterns that exist. The Corrector's failure to learn any useful mapping from noisy $\hat{z}$ to clean z is direct evidence that JPEG compression errors in this model's latent space have no consistent statistical structure - they are genuinely random across bit positions. There is no fingerprint to learn.

Finding: JPEG errors in this latent space are statistically random with no learnable structure.

This is the most significant analytical finding of the error correction investigation. Four independent approaches - parity-based, soft-decision, rule-based redundancy, and machine learning - each failed for a different reason, and each failure points to the same conclusion: JPEG compression errors in this latent space are random and unstructured.

5.5 Phase 5: 3x Bit Repetition - The Winning Approach

Hypothesis: If errors are genuinely random and unstructured, brute-force redundancy should work better than any structured approach. Encoding each bit three times and recovering via majority vote does not require errors to be isolated, structured, or predictable - it simply exploits the fact that random errors at a 13–17% rate are unlikely to corrupt two out of three copies simultaneously.

Tradeoff: Message capacity drops from ~12 to ~4 ASCII characters. But the hypothesis is that majority voting's tolerance for randomly distributed errors will produce dramatically better exact match rates.

Condition	Standard	Hamming	Confidence	Learned Denoiser	3x Repetition
No compression	86.19% / 0%	85.75% / 2%	86.19% / 0%	85.44% / 0%	94.75% / 26%
JPEG QF=75	86.56% / 0%	84.56% / 0%	86.56% / 0%	86.06% / 0%	94.50% / 24%
JPEG QF=30	83.88% / 0%	78.69% / 0%	83.88% / 0%	83.56% / 0%	92.50% / 10%

Format: bit accuracy % / exact match %. 3x Repetition won every condition by a wide margin, achieving up to 26% exact message recovery - meaning more than one in four 4-character messages recovered perfectly after JPEG compression. No other approach produced meaningful exact match rates.

The result is decisive. Repetition coding is the correct error correction approach for this specific system because the underlying error distribution is random - exactly the condition where majority voting excels and where structured approaches fail.

---

6. Analysis

6.1 Why Each Approach Succeeded or Failed

The five-phase investigation produced a clear picture of the error landscape in this system's latent space under JPEG compression:

- Hamming(7,4) failed: Assumes errors are sparse and isolated within 7-bit blocks. JPEG produces dense, distributed errors that violate this assumption and cause miscorrection.
- Confidence weighting failed: Assumes the Decoder produces low-confidence outputs on error-prone bits. This Decoder's high-confidence output profile provides no signal for soft-decision strategies.
- Learned Denoiser failed: Assumes errors have learnable statistical structure. Corrector's inability to learn above 74.45% validation accuracy after 50 epochs on 20,000 pairs confirms errors are structurally random.
- 3x Repetition succeeded: Makes no assumptions about error structure. Majority voting exploits only the error rate (13–17%), not any pattern, and at that rate the probability of majority corruption is low enough to produce meaningful exact recovery.

6.2 The Uncompressed Performance Tradeoff

The compression-aware model's drop from 100% (baseline) to 83% uncompressed accuracy is the expected consequence of JPEG-aware training. The Decoder was forced to abandon fine-grained high-frequency pixel features in favor of compressionrobust lower-frequency patterns. This is the correct engineering tradeoff: a system with 83% accuracy under all real-world conditions is more useful than one with 100% in a lab and 58% in the field.

6.3 Compression Generalization

The model was trained at QF=75 and tested at QF=30 - a significantly more aggressive compression level. Maintaining 80%+ accuracy at QF=30 demonstrates that the learned compression resistance generalizes across quality levels, suggesting the model learned properties of JPEG compression in general rather than artifacts of one specific quality setting.

6.4 The Significance of the Learned Denoiser's Failure

The Corrector's failure to learn is not merely a negative result - it is an informative one. It confirms that there is no hidden structure in the JPEG error distribution that a more sophisticated approach could exploit. This finding strengthens confidence in 3x repetition as the appropriate solution: if a 600,548-parameter neural network trained on 20,000 examples cannot find a pattern, no deterministic algorithm will either. Redundancy is the right tool precisely because there is no pattern to exploit.

---

7. Observations

- Compression-aware training is the necessary foundation. Without it, no error correction scheme can compensate for the baseline's 58–59% collapse under JPEG. The compression-aware model's 80–83% bit accuracy is what makes any subsequent error correction viable.
- Structured error correction is inappropriate for this error channel. Hamming, confidence weighting, and learned denoising all assume structure that does not exist in JPEG's damage to this latent space. Testing all three and observing distinct failure modes for each strengthens this conclusion.
- 3x Repetition is the optimal approach for this system as currently trained. Majority voting requires only that errors be random and occur at a manageable rate - both conditions the system satisfies.
- The Edison principle applies. Each failed approach narrowed the solution space and contributed to understanding. The final answer - that JPEG errors here are random and unstructured - is supported by four independent experimental lines of evidence.
- Bit-level accuracy is the correct primary metric. Character-level exact match is too strict for evaluating bit accuracy improvements, while too lenient for evaluating practical usability. Both should be reported together, as done throughout this investigation.
- The Learned Denoiser's 74.45% validation accuracy reveals a fundamental property of the system. Any future work on this architecture should assume random error distribution and design accordingly.

---

8. Next Steps and Future Work

- Reed-Solomon error correction: Reed-Solomon can correct multiple errors per block and is used in CDs, QR codes, and deep-space communications. Given

that Hamming failed due to multi-bit errors per block, Reed-Solomon is the logical next structured approach to evaluate.

- Compression-aware training at QF=30: Retraining with compression simulation at QF=30 rather than QF=75 would directly reduce the bit error rate at aggressive compression, potentially bringing it below the threshold where even Hamming becomes effective.
- Variable QF training: Randomly sampling QF from 30 to 90 during each training batch may produce better generalization across compression levels than fixedQF training.
- Repetition + compression training synergy: Testing whether further reducing the bit error rate through retraining improves repetition coding's exact match rate beyond the current 26% ceiling.
- Domain generalization: Training on datasets beyond CelebA (landscapes, objects) would make generated cover images less conspicuous in contexts where a stream of face images would arouse suspicion.
- Photo-resilience testing: Evaluating whether messages survive being photographed from a screen - the analog hole - would test a real-world attack scenario not covered by the current suite.

Acknowledgments

The author wishes to thank Professor Stephen Dennis for his patience, his willingness to extend the opportunity to complete this course, and for the time and encouragement that made the additional research in this report possible. A professor who motivates a student to go further than originally planned is one of the rarest and most valuable things in education.

References and AI Assistance Disclosure

References

[1] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. arXiv:1511.06434.

[2] Zhang, K. A., Cuesta-Infante, A., Xu, L., & Veeramachaneni, K. (2019).
SteganoGAN: High Capacity Image Steganography with GANs. arXiv:1901.03892.

[3] Liu, Z., Luo, P., Wang, X., & Tang, X. (2015). Deep Learning Face Attributes in the Wild (CelebA Dataset). ICCV 2015.

[4] Goodfellow, I. et al. (2014). Generative Adversarial Nets. NeurIPS 2014.

[5] Base repository (forked baseline): DCGAN-Based-Steganography-Project by Zain. GitHub, 2024.

AI Assistance Disclosure

Error correction codec implementations (hamming_codec.py, repetition_codec.py, confidence_decoder.py, hamming_compressiontest.py, threeway_compressiontest.py, fourway_compressiontest.py, fiveway_compressiontest.py, generate_training_pairs.py, train_corrector.py) were developed with the assistance of Claude (Anthropic), an AI assistant, and subsequently reviewed, tested, and integrated by the author. The technical report document was drafted with AI writing assistance and edited by the author. All experimental hypotheses, training runs, test execution, result interpretation, and analytical conclusions are the author's own work.